BattleOps Week 4 Modules

OBJECTIVE: Operate observability components under load, debug systems using metrics, logs, and traces, apply SLO-driven production thinking, and automate healing logic.

MONDAY: Prometheus / Grafana Under Load

CRITICAL CONCEPTS

- High-cardinality metrics diagnosis
- Recording rules for performance
- Prometheus federation patterns
- Grafana alert tuning
- Metric-based incident debugging (CPU throttling, memory leaks)

KEY OUTCOMES

- You can debug a node/pod using only metrics
- You understand when metrics are lying (high cardinality traps)
- You can design production dashboards that avoid noise

WHAT TO PRACTICE

Lab 1: Break Prometheus with High-Cardinality Metrics

Deploy a pod that exports metrics like:
request_duration_seconds_bucket{user="random-uuid"}

This will flood Prometheus.

Your tasks:

- Observe target load
- Identify cause via /api/v1/status/tsdb
- Fix by dropping labels using relabel configs

Lab 2: Create Recording Rules

Create a rule: node: cpu_usage:avg5m = avg(rate(node_cpu_seconds_total[5m])) by (instance)

Observe query latency improvement in Prometheus UI.

Lab 3: Build a War-Room Dashboard

Panels required:

- CPU throttling % per container
- Memory working set vs limit
- Pod restarts (over 24h)
- API server request rate / saturation
- etcd disk IO latency

This is a *production-grade troubleshooting dashboard*.

WHERE TO STUDY

- Prometheus TSDB internals docs
- Grafana dashboards best practices
- Robust Perception blog (Prometheus maintainers)

BATTLEOPS MINDSET NOTES

- High cardinality is the **#1 silent killer** in production Prometheus.
- Dashboards are not decoration — they are **operating panels**.
- Always reduce data before visualizing it.

TUESDAY: Loki + FluentBit Failure Modes (Logs at Scale)

CRITICAL CONCEPTS

- Log ingestion backpressure
- FluentBit memory + CPU limits
- Loki compactor & querier bottlenecks
- LogQL filters for debugging incidents
- Misconfigured pipelines & dropped logs

KEY OUTCOMES

- You can detect log ingestion failures
- You know how to scale Loki components
- You can debug out-of-order logs and noisy log streams

WHAT TO PRACTICE

Lab 1: Create a Logging Storm

Deploy a pod generating 2,000 logs/sec: kubectl run log-bomber --image=busybox \--sh -c "while true; do echo $(date) fire; done"

Observe:

- FluentBit CPU
- Loki ingestion throughput
- Dropped logs

Lab 2: Locate the Bottleneck

Use: kubectl top pods -n observability
Grafana → Loki metrics dashboards

Check:

- loki_ingester_lines_received_total
- loki_ingester_chunks_stored_total
- loki_distributor_bytes_received_total

Lab 3: Build a LogQL Incident Query

Requirements:

- Filter only ERROR logs
- Detect 5xx patterns

- Aggregate by pod

Example: {app="payment"} |= "ERROR" | unwrap timestamp | sort_desc

WHERE TO STUDY

- Loki Architecture Docs
- FluentBit Pipeline Deep Dive
- Grafana Tempo + Loki integration guide

BATTLEOPS MINDSET NOTES

- Logs lie unless you structure them.
- Logging pipelines must be **predictable**, not “verbose.”
- If logging stops → your observability is blind.

WEDNESDAY: Distributed Tracing Under Stress (OpenTelemetry + Tempo)

CRITICAL CONCEPTS

- Sampling strategies
- Tracing cost optimization
- High latency span detection
- Service-to-service flamegraphs
- Missing spans & broken context propagation

KEY OUTCOMES

- You can recreate latency issues using traces
- You can detect which microservice is the root cause
- You understand sampling → cost → accuracy

WHAT TO PRACTICE

Lab 1: Break Trace Sampling

Set sampling to 100% → force high load.
Expected: tracing pipeline slowdown.
Then fix by:

- switching to parent-based sampling
- adding sampler limits

Lab 2: Identify Deep Latency

Use Explore → Traces in Grafana:
Look for:

- highest p99 span
- longest internal call
- slow DB calls
- context propagation failures

Lab 3: Trace Debug War-Room Simulation

We give students a broken microservice chain: frontend → cart → checkout → payment

Your tasks:

- Find which service is slow
- Identify the exact span
- Provide RCA with span timeline

WHERE TO STUDY

- OpenTelemetry Sampling Docs
- Google Dapper Paper
- CNCF Observability Whitepaper

BATTLEOPS MINDSET NOTES

- Tracing is your **lie detector** — it never hides where the latency is.
- Sampling wrong = losing the incident.

THURSDAY: Autoscaling War-Room (HPA, VPA, KEDA Failures)

CRITICAL CONCEPTS

- Throttling + container limits
- Out-of-sync metrics server
- HPA lag, oscillation, cooldown windows
- VPA eviction logic
- KEDA event-driven anomalies

KEY OUTCOMES

- You know how to debug why scaling DIDN'T happen
- You know how to fix false positives / oscillations
- You know how to simulate real-world scale bursts

WHAT TO PRACTICE

Lab 1: Break the HPA

Deploy a workload with CPU limits LOWER than required: → HPA reads wrong metrics → no scaling occurs.

Student must:

- find throttling
- fix limits
- validate HPA reaction time

Lab 2: Simulate Sudden Traffic Spike

Use hey or wrk: hey -z 30s -q 20 http://autoscale-demo
Observe:

- time for HPA to react
- pod creation cold start
- recovery patterns

Lab 3: KEDA Event Misfire

Deploy KEDA ScaledObject but misconfigure trigger threshold.

Student must fix:

- incorrect queueLength
- cooldown period
- missing role permissions

WHERE TO STUDY

- KEDA Scaling Patterns
- Kubernetes Autoscaling Best Practices
- GKE / EKS HPA internals

BATTLEOPS MINDSET NOTES

- Scaling issues break production more than outages.
- Autoscaling MUST be predictable and observable.

FRIDAY: SLOs, Alerting, Incident Response Simulation

CRITICAL CONCEPTS

- Error budgets
- Multi-window burn rates
- SLO-driven alerting
- Playbooks & paging policies
- Incident prioritization system

KEY OUTCOMES

- You can write production-grade SLOs
- You understand alert fatigue & how to avoid it
- You can run a live war-room simulation

WHAT TO PRACTICE

Lab 1: Create a 4-Golden-Signals Dashboard

Must include:

- Latency p50/p90/p99
- Error rate
- Throughput
- Saturation (CPU/Memory/Queue)

Lab 2: Burn Rate Alerts

Configure: 2% error budget burned in 1 hour → page SRE
5% burned in 24 hours → create ticket

Lab 3: Simulated Incident

Students receive:

- Logs
- Metrics
- Traces
- Alerts
- User complaint

They must produce:

- SLO violation analysis
- Root cause
- Remediation plan

- Prevention strategy

WHERE TO STUDY

- SRE Workbook (Google)
- Alerting Golden Rules (Rob Ewaschuk)
- Nobl9 SLO Academy

BATTLEOPS MINDSET NOTES

- Only SLOs define “pain.”
- Alerts DRAIN engineers unless intentional.
- SREs must detect the impact **before users complain**.